AI Developer Candidate Assignment

Travel Reimbursement Approval Agent

Purpose: Assess hands-on ability to build a practical GenAI and Agentic AI solution, not just describe one.

Business scenario	Review employee travel reimbursement claims against policy, receipts, limits, and approval rules.
Build objective	Create a working AI-assisted agent that evaluates a claim and returns a structured recommendation.
Expected effort	Designed to be completed in 2-3 days using free-tier, open-source, or local tools.
Expected outcome	Runnable prototype, sample inputs/outputs, and a short explanation of design choices and trade-offs.

1. Assignment Overview

Build a working prototype of a Travel Reimbursement Approval Agent. The agent should accept a claim, use policy/rule context and tools, and produce a clear decision such as Approve, Partially Approve, Reject, or Manual Review. The emphasis is on clean implementation, GenAI reasoning, tool usage, and reliability in ambiguous cases.

2. Scope and Constraints

- Keep the solution practical and lightweight; do not build a production-grade enterprise system.
- Use Python, JavaScript/TypeScript, or another language you are comfortable with.
- You may use LangChain, LlamaIndex, Semantic Kernel, CrewAI, AutoGen, direct LLM APIs, or an equivalent lightweight approach.
- Use mock/sample policy documents, claims, receipts, and approval rules. No real employee or company data should be used.
- A simple CLI, notebook, API endpoint, or small UI is sufficient if the demo is easy to run and understand.

3. Minimum Functional Expectations

- **Claim intake:** Accept a reimbursement claim from JSON, CSV, form input, or API request.
- **Context grounding:** Retrieve or reference relevant policy/rule context before making a decision.
- **Tool/function usage:** Use at least two meaningful tools or functions, such as policy lookup, receipt completeness check, per-diem/limit checker, duplicate detector, approval threshold check, or output validator.
- **GenAI/Agentic workflow:** Show how the LLM decides when to use tools, combines results, and handles missing or conflicting information.
- **Structured output:** Return consistent JSON or table output with decision, approved amount, deductions/rejected amount, missing documents, policy references or rule basis, confidence, and short explanation.
- **Manual review handling:** Route uncertain, incomplete, or policy-exception cases to Manual Review rather than forcing a decision.

4. Suggested Demo Inputs

- A short travel policy file in Markdown, TXT, PDF, or JSON.
- A small limit table, approval matrix, or category eligibility file.
- Three to five sample reimbursement claims covering approved, partially approved, rejected, and manual review cases.
- Optional mock receipts or receipt metadata, such as date, amount, category, vendor, and attachment status.

5. Deliverables

- **Runnable code:** A Git repository, zip file, or shared folder with source code and sample data.
- **README:** Setup steps, required environment variables, how to run the demo, and key design choices.
- **Sample outputs:** At least three example claims with generated decisions and explanations.
- **Demo evidence:** Screenshots, API responses, notebook output, or a short walkthrough. **Be prepared to demonstrate the prototype during the interview.**
- **Assumptions and limitations:** Briefly list assumptions, simplifications, known gaps, and what you would improve next.

6. Evaluation Criteria

- Hands-on implementation quality: runnable, understandable, and reasonably organized code.
- Practical use of GenAI and Agentic AI: tool calling, workflow control, prompting, and context handling.
- Business correctness: sensible reimbursement decisions grounded in policy/rule context.
- Reliability: structured outputs, validation, fallbacks, and manual review for uncertain cases.
- Developer judgement: clear trade-offs, simple design, and avoidable over-engineering removed.

7. Optional Enhancements

- Simple UI or chatbot interface.
- Audit trail showing retrieved context, tools called, and intermediate checks.
- Basic test cases or evaluation script for sample claims.
- Confidence score or reason codes for manual review.
- MCP-based tool integration, if you are comfortable implementing it within the timebox.

Submission guidance: This assignment is intentionally timeboxed to 2-3 days. A small, working, well-explained prototype is preferred over a broad but fragile implementation.